Benchmarking the Full Pipeline of Materialized-View-Based Query Rewriting

Xinjie Hu
Simon Fraser University
Burnaby, Canada
xha102@sfu.ca

Zhengjie Miao
Simon Fraser University
Burnaby, Canada
zhengjie@sfu.ca

ABSTRACT

Materialized views (MVs) accelerate OLAP and data-warehouse workloads by precomputing reusable subexpressions, but practical MV-based query acceleration is a multi-stage pipeline: candidate enumeration, view selection under storage budgets, and query rewriting inside the optimizer. Existing evaluations typically study only parts of this pipeline and within a single system, leaving end-to-end trade-offs and cross-system behavior unclear.

In this paper, we benchmark MV-based query rewriting by jointly evaluating enumeration, selection, and rewriting with a modular evaluation framework and by using controlled ablations. We also introduce a cross-engine protocol allowing us to compare systems that expose only execution plans by contrasting native optimizer-level rewriting with portable SQL rewriting baselines when available. Across representative academic methods and modern open-source and commercial systems, we find strong interaction effects across stages and large variability in MV usage and realized savings. We identify recurring failure modes that explain performance regressions after rewriting. Our results highlight which pipeline stages most often limit performance and provide evidence to guide future MV enumeration, selection, and rewriting designs.

PVLDB Reference Format:
Xinjie Hu and Zhengjie Miao. Benchmarking the Full Pipeline of Materialized-View-Based Query Rewriting. PVLDB, 19(11): 3772 - 3785, 2026.
doi:10.14778/3836663.3836724

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/edx-h/Benchmarking-MV-Based-Rewriting.

1 INTRODUCTION

As analytical workloads grow in scale and complexity, many queries repeatedly execute expensive subexpressions (e.g., joins and aggregations) over large base tables, leading to high latency and substantial redundant computation. Materialized views (MVs) offer a principled way to eliminate this redundancy: by precomputing and storing reusable subexpressions, a DBMS can rewrite incoming queries to reuse materialized results, replacing repeated computation with a (typically cheaper) view scan. This motivates MV-based query rewriting, which is now supported in an increasing number of database systems (automatic MV rewrite support: Oracle 2009, Snowflake 2018, Redshift 2020, BigQuery 2021, StarRocks 2023, Doris 2024 [1, 3–7]).

Conceptually, MV reuse is straightforward, whereas realizing MV-based rewriting in practice requires a multi-stage pipeline whose components depend on one another: given a workload, an *enumerator* generates candidate views; a *selector* chooses a subset under a storage budget to maximize expected runtime savings; and a *rewriter* decides whether and how to rewrite each query to use the selected views. In practice, production systems differ in what they expose: some expose intermediate outputs such as selected views, while others do not; some expose rewritten SQL, while others perform rewriting internally and only reveal the final execution plan. This limited observability makes it difficult to compare systems — especially rewriting behavior across engines — and motivates our cross-platform evaluation protocol.

Existing research targets the pipeline asymmetrically: enumeration [9, 15, 29] and selection [26, 31, 40, 54] have received dedicated empirical study and benchmark-style cross-method comparison, whereas the rewriting stage has been studied algorithmically [41, 46–49]. But to our knowledge, no comprehensive benchmark study compares industrial rewriters across engines under a common protocol. This leaves practitioners without empirical guidance on the most engine-dependent stage; combined with the absence of an end-to-end evaluation across all three stages under a unified setting, it makes pipeline behavior and practical trade-offs difficult to assess. To fill this gap, we build a unified evaluation protocol that covers all three stages — enumeration, selection, and rewriting — and introduce a cross-platform protocol to support both pluggable components and systems with limited observability. In this paper, we experimentally investigate the full MV-based rewriting pipeline to answer the following research questions:

RQ1: Is MV-based query rewriting "solved" in practice? Do end-to-end pipelines deliver *consistent* workload-level savings across workloads and budgets? How often do rewrites regress, and how do *industrial end-to-end solutions* compare with *modular baselines*?

RQ2: How does MV-based query rewriting vary across engines under limited observability? When engines expose only plan-level output rather than rewritten SQL, how can we compare their MV exploitation fairly? Under a common MV set, how does native optimizer-level rewriting compare with a portable SQL rewrite baseline executed on the same engine?

RQ3: Which stage is the bottleneck for end-to-end performance, and why? When is the limiting factor candidate generation, budget-constrained selection, or the rewriter's ability to exploit

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 19, No. 11 ISSN 2150-8097.
doi:10.14778/3836663.3836724

selected views? What workload and candidate-space characteristics explain these bottlenecks, and what failure modes recur?

Contributions. We provide a systematic evaluation of MV-based query rewriting methods, covering both end-to-end pipelines and individual components. Our contributions are:

(1) **End-to-end empirical evaluation across all three stages.** We develop a modular framework that decomposes MV-based query rewriting into enumeration, selection, and rewriting, enabling systematic comparison and controlled ablations (fix two stages, vary the third). To our knowledge, this is the first end-to-end empirical study that jointly evaluates representative methods across all stages under a unified protocol (Section 3).
(2) **Cross-engine protocol and a systematic comparison of industrial and open-source rewriters.** We study a broad set of open-source and commercial systems and propose a cross-engine evaluation protocol that incorporates systems with limited observability (those exposing only plans rather than rewritten SQL) by comparing rewriting on a common execution engine. Using it, we provide the first benchmark comparison of industrial rewriters — `HIV`, `CAL`, `DRS`, `STR`, `Sys-A`, `Sys-B`, and `Sys-C` — under a common protocol with shared workloads and view sets, on rewrite success rate, runtime impact, and failure-mode signatures (Sections 3, 7, 9.3, 9.4).
(3) **Reusable implementation and metrics.** We implement and integrate representative baselines across stages and systems behind a unified interface for plugging in additional methods and engines, and curate stage-wise and end-to-end metrics (e.g., workload time saving, per-view query coverage, regression behavior) under a standardized protocol (Sections 4–7).

Summary of findings. Our results show that isolated stage-wise evaluations are insufficient for understanding MV-based rewriting pipelines. Pipeline outcomes depend on how candidate generation, selection under budget, and rewriting interact. We summarize three quantitative, workload-conditioned findings.

- **Single-stage rankings can be locally valid but globally misleading.** On PostgreSQL/JOB/`HIV` rewriter at 1 GB, fixing `BigSubs` makes `ECSE` outperform `Basic` by 43.85 pp, while fixing `Basic` makes `GnnMV` outperform `BigSubs` by 47.53 pp. Yet both `ECSE–BigSubs` and `Basic–GnnMV` are high-performing pipelines (Figure 3). Thus, enumerators and selectors cannot be ranked independently: different candidate spaces interact with different selectors (§ 9).
- **The dominant bottleneck changes with workload and pipeline context.** Enumeration limits performance when high-coverage views are missing (Figure 3, §9.1); selection dominates when such views must be ranked under tight budgets (Table 29 in the supplementary material); and rewriting dominates when selected views are not exploited by the engine (Table 4, §9.3). These shifts explain why single-stage evaluation under one fixed context misses important failure modes.
- **Modular pipelines often beat commercial integrated systems, but not uniformly.** On JOB, the best modular pipelines outperform commercial auto-selection + native rewriting by roughly 40 pp or more (Figure 4); on TPC-DS, the gap narrows or reverses when the integrated enumeration/selection in the commercial system produces more compact, useful views (through column pruning) that fit into the storage budget (Section 10).

Outline. Section 2 defines the MV-based rewriting pipeline. Section 3 presents our modular evaluation framework and cross-engine protocol. Sections 4–6 describe the evaluated enumerators, selectors, and rewriters. Section 7 gives experimental setup and metrics. Section 8 shows end-to-end and cross-engine results, establishing why isolated stage rankings are insufficient. Section 9 then diagnoses the mechanisms behind these results through stage-wise and robustness analyses. Section 10 provides representative case studies, and Section 11 provides practical guidance and future directions. Section 12 concludes our research. Throughout the paper, "supplementary material" refers to the full technical report[1], containing an appendix with additional details.

2 PRELIMINARIES AND BACKGROUND

Given a set of queries $\mathcal{Q} = \{Q_1, \ldots, Q_m\}$, a database schema S, and a database instance D of S, the goal of MV-based query rewriting is to accelerate the execution of queries in $\mathcal{Q}$ using materialized views; the task is further decomposed into three subtasks.

Query Rewrite. Given a query Q, a database instance D of S, a set of views $\mathcal{V} = \{\mathtt{V}_1, \mathtt{V}_2, \ldots, \mathtt{V}_n\}$, views in $\mathcal{V}$ are instantiated as materialized views $\mathcal{M} = \{\mathtt{M}_1 = \mathtt{V}_1(D), \mathtt{M}_2 = \mathtt{V}_2(D), \ldots, \mathtt{M}_n = \mathtt{V}_n(D)\}$, the query rewrite problem is to find a rewritten query $\mathtt{R}$ of Q with the minimum $\mathrm{cost}(\mathtt{R}|D, \mathcal{M})$, where $\mathrm{cost}(\mathtt{R}|D, \mathcal{M})$ is the evaluation cost of $\mathtt{R}$ over the database instance D with views materialized.

View Selection. Given $\mathcal{Q} = \{Q_1, \ldots, Q_m\}$ and a set of materialized views $\mathcal{M} = \{\mathtt{M}_1 = \mathtt{V}_1(D), \mathtt{M}_2 = \mathtt{V}_2(D), \ldots, \mathtt{M}_n = \mathtt{V}_n(D)\}$, for any non-empty subset of $\mathcal{V}$, the view selection problem is to find a subset of MVs $\mathcal{M}^* \subseteq \mathcal{M}$ to minimize the cost of evaluating queries in $\mathcal{Q}$ within a storage budget constraint:

$$\mathcal{M}^* = \operatorname*{argmin}_{\mathcal{M}' \subseteq \mathcal{M}, \mathrm{size}(\mathcal{M}') \le B} \sum_{i=1}^{m} \mathrm{cost}(\mathtt{R}_i|D, \mathcal{M}')$$

where $\mathtt{R}_i$ is a rewrite of query Q_i, and $\mathrm{cost}(\mathtt{R}_i|D, \mathcal{M}')$ denotes the execution cost of $\mathtt{R}_i$ on D when the MVs in $\mathcal{M}'$ are available, and $\mathrm{size}(\mathcal{M}')$ is the storage cost under budget B.

Candidate View Enumeration. Given $\mathcal{Q} = \{Q_1, \ldots, Q_m\}$ and S, candidate view enumeration (enumeration for short) is a procedure that outputs a finite set of view definitions $\mathcal{V} = \{\mathtt{V}_1, \ldots, \mathtt{V}_n\}$ to be considered by the selector. In practice, $\mathcal{V}$ is often derived from query subexpressions (e.g., joins/aggregations) and may apply pruning rules to control the candidate set size.

3 MODULAR EVALUATION FRAMEWORK

We design a modular evaluation framework (Figure 1). Given a query workload, an *enumerator* generates a set of candidate views. A *selector* chooses a subset of candidates under a storage budget. Finally, a *rewriter* rewrites workload queries by substituting view scans for matching subexpressions, returning either rewritten SQL (when exposed) or an execution plan.

Pluggability. We call a pipeline component *pluggable* if it exposes its intermediate outputs through a common interface (e.g., candidate

[1] https://arxiv.org/abs/2607.19679v1

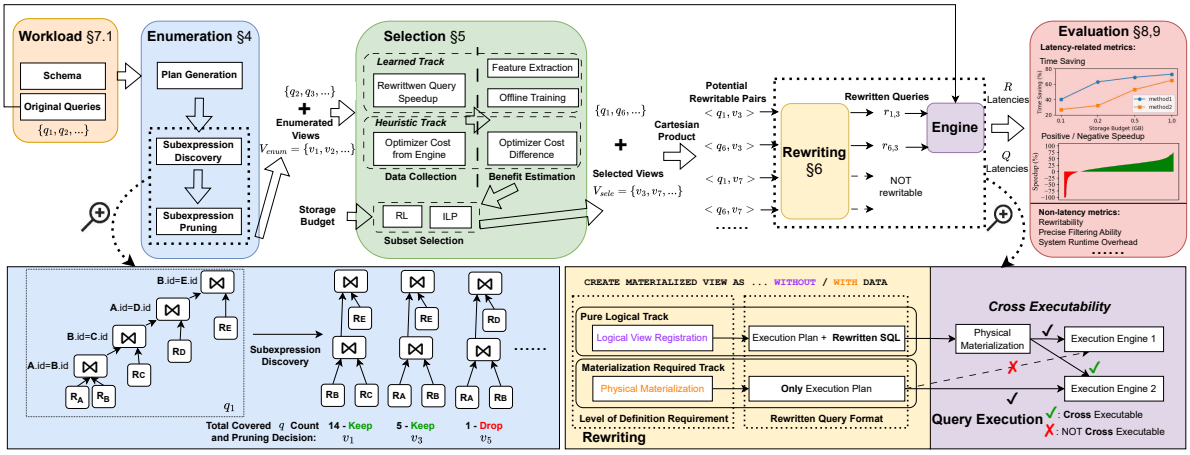


Figure 1: The modular evaluation framework for MV-based query rewriting pipeline. Blocks at the bottom show the details of the components of the same color in the framework. Numbers denote the corresponding section.

views, selected views, or explicit rewrites), allowing us to substitute alternative implementations without modifying the rest of the pipeline. In practice, pluggability differs across stages: most enumerators and recommenders are pluggable by exposing view definitions and/or selected views, whereas most rewriting is performed only in a *plan-transparent* way, i.e., merely observable through the final `EXPLAIN` plan rather than an explicit rewritten SQL query. We determine rewrite success by checking whether the rewriter returns a rewritten SQL or the plan references a materialized view.

Cross-platform evaluation. For fully pluggable pipelines, our framework enables evaluation over combinations of methods, i.e., $\{Enumerators\}\times\{Selectors\}\times\{Rewriters\}$. For pipelines with non-pluggable components (most commonly plan-transparent rewriters), we use a cross-platform protocol. At a high level, we evaluate optimizer-driven rewriting on the target engine through plan inspection and runtime, and additionally execute portable rewritten SQL produced by a SQL-transparent rewriter on the same engine. Details of the cross-platform protocol are provided in Section 7.

4 CANDIDATE VIEW ENUMERATION

4.1 View Enumeration Methods

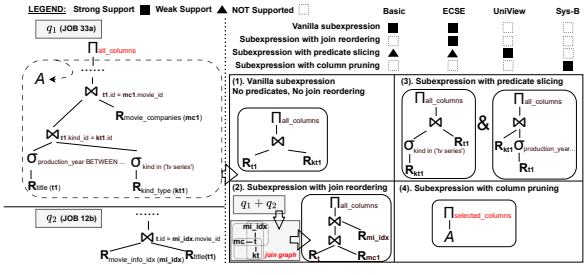


Figure 2: Behavior Comparison of Enumerators. We take 33a and 12b from JOB as two original queries for illustration.

Candidate view enumeration generates candidate view definitions from a workload. Prior approaches can be organized by how they define and prune the candidate space. Classical approaches enumerate structured search spaces such as data-cube lattices or AND/OR view graphs [22, 23, 28, 29]. Workload-subexpression approaches [9, 18, 19, 31–33, 55] extract candidates from query plans or shared subplans, often with pruning based on frequency, cost, or containment relationships. Join-graph approaches further expand the candidate space by deriving workload-level join structures beyond a single optimizer plan [10]. MVPP-style approaches merge workload queries into a shared DAG and select reusable intermediate nodes as candidates [26, 30, 53, 54]. Finally, production systems may couple enumeration with selection and expose only the resulting recommended views. The supplementary material provides a more detailed survey; here we focus on the representative enumerators used in our evaluation (Table 1).

4.2 Evaluated Methods and Implementation

We evaluate the four representative enumerators in Table 1 (three academic methods plus the commercial `Sys-B`). The evaluated enumerators differ along three axes: join-structure expansion, predicate slicing, and column projection (Figure 2); we describe each in two to three sentences below and leave the detailed per-query behavior to the figure.

`Basic` [28, 51] is a classical heuristic enumerator that builds candidates bottom-up from the query plan, adding one join relation per

Table 1: Evaluated enumerators and their taxonomy.

Method	Category	Key behavior	Why included
`Basic`	Plan-subtree	Bottom-up plan subexpressions	Classical heuristic baseline
`ECSE`	Join-graph / workload	Explores join-order variants	Tests structural expansion
`UniView`	MVPP	Predicate-rich merged DAG views	Recent open-source method
`Sys-B`	Industrial integrated	Coupled enum./selection	Production system behavior

Table 2: Evaluated selectors.

Method	Benefit model	Subset search	Why included
BigSubs	Optimizer-cost proxy	ILP	Classical cost-based method
GnnMV	Learned utility	ILP	Learned ranking with budgeted selection
UniView	Learned cost model	RL/DQN	Recent open-source learned selector
Sys-B	System-internal	System-internal	Representative industrial selector

step, so every candidate is a subtree of the chosen plan. It preserves all columns and slices predicates only conservatively (after all relations are joined) to limit candidate explosion, and consequently cannot discover join orders absent from that plan.

ECSE [10] expands the candidate space by discovering join patterns shared across queries but absent from any single plan [10, 19, 40, 42, 44, 52]: it maintains a workload-level join graph and takes the transitive closure over equality predicates to synthesize new join structures. Its predicate and projection handling follows Basic.

UniView [53], the only open-source enumerator we found, represents MVPP-style methods [26, 30, 54]: it merges the workload's queries into a single DAG whose non-leaf nodes become candidates. Its implementation slices predicates so that every candidate carries at least one predicate (predicate-free views are pruned).

Sys-B couples enumeration with selection inside a commercial engine, generating candidates internally; we characterize its aggressive column pruning in the case study in Section 10.

5 VIEW SELECTION

5.1 View Selection Methods

View selection chooses a subset of candidate MVs under a storage budget. Prior work differs mainly in how it estimates view benefit and how it searches the budget-constrained subset space. Classical methods rely on optimizer cost estimates and select views using greedy, dynamic-programming, or integer-programming algorithms over structured candidate spaces such as cube lattices or AND/OR view graphs [22, 23, 28, 31, 43]. Other approaches use metaheuristics such as genetic search, represented by MVPP [54]. More recent learned methods, including DQM [40], AutoView [26], and UniView, learn query/view representations and cost or benefit models, sometimes combined with reinforcement learning [26, 40, 53]. Commercial systems such as Sys-B further couple enumeration and selection into an integrated black-box recommender. Table 2 maps these categories to the selectors evaluated in this paper.

5.2 Evaluated Methods

MV selection can be divided into two sub-stages: estimating the benefit of each candidate view and selecting a subset under the storage budget. We evaluate four representative selectors, including one commercial integrated system. Table 2 summarizes their benefit model, subset-search strategy, and role in our evaluation.

BigSubs [31] formulates selection as an ILP that maximizes the total cost-proxy utility of selected views (per-query reduction $\text{cost}(q) - \text{cost}(q \mid v)$) under the budget, with overlap constraints discouraging redundant views (full formulation in the supplementary material).

GnnMV [25] replaces the cost proxy with a graph neural network (GNN) that predicts per-view utility from the structure of queries, candidate views, and base tables, then selects under budget with ILP. The model is trained offline using features extracted from query workloads, including query structure statistics and estimated costs.

UniView [53] uses a deep neural network (DNN) to estimate post-rewrite execution cost and a Deep Q-Network (with reinforcement learning) to choose the view set.

Sys-B couples enumeration and selection in the backend and exposes only the selected views, with an undisclosed internal benefit model. By parsing historical query logs, it identifies recurring subplans to extract structured SPJG patterns.

Implementation. Most prior MV-selection systems are not publicly available. We therefore implement BigSubs using its ILP formulation and implement GnnMV following the model and training procedure described in its paper. For UniView, we use its public implementation. The learning-based selectors are trained separately for each dataset using the corresponding training workload. For Sys-B, we use the system-provided recommender and treat its internal selection logic as a black box.

6 REWRITER

6.1 Rewriting Queries Using Views

Query rewriting using views has a long history in data warehouses and data integration systems [24, 28, 35, 38]. Classical work studies rewriting through query containment and equivalence, first for conjunctive queries and later for richer SQL fragments with aggregation, comparison predicates, and set operations [13, 16, 20, 21, 37, 45]. These extensions substantially increase the complexity of deciding whether and how a query can be rewritten using available views [8, 24]. Modern DBMSs typically implement MV rewriting inside the query optimizer rather than as a standalone rule system [12, 14, 50]. Under this architecture, view substitutions are considered as part of plan search and interact with normalization, rule matching, and cost estimation. Despite this mature algorithmic and system literature, to our knowledge no prior study systematically benchmarks practical MV rewriters across engines under a common protocol. This motivates our evaluation of both SQL-transparent and plan-transparent rewriters under a common cross-engine protocol and the cross-engine analysis in Section 9.3.

6.2 Evaluated Methods

Most academic rewriting algorithms are either conceptual or not publicly available as robust SQL-level tools. The only open-source academic system we found with an implemented rewriting component is UniView, but in our preliminary validation it produced a non-negligible number of non-equivalent rewrites, so we exclude it from the rewriter comparison and focus on systems whose rewrites are validated through production query optimizers. We evaluate seven practical rewriters spanning SQL-transparent and plan-transparent systems.

CAL (vanilla Calcite [11]) applies the built-in SubstitutionVisitor class and default view-related rules over relational-algebra plans, with no engine-specific tuning.

HIV (Apache Hive [2]) embeds Calcite-based view rewriting directly inside its optimizer and, with CAL, is one of the two rewriters that emit portable rewritten SQL.

STR (StarRocks [7]) and **DRS** (Doris [1]) are open-source MPP engines with native optimizer-level MV rewriting; STR documents that its design is inspired by prior work on optimizing queries using materialized views [19][2].

Sys-A, **Sys-B**, and **Sys-C** are commercial cloud systems that provide an automatic query rewrite feature inside the optimizer; their optimizer-level MV rewriting is exposed through execution plans rather than rewritten SQL. We treat them as plan-transparent rewriters in our cross-engine protocol.

Cross-engine executability. Only CAL and HIV expose rewritten SQL; these rewrites can be executed on other engines after dialect translation when the translated SQL preserves semantics (we use sqlglot in our implementation). The remaining rewriters are plan-transparent: they expose only the final execution plan, and we infer rewrite success when the plan references a materialized view. This observability gap motivates our cross-engine protocol (Section 7): for each plan-transparent engine, we compare its native optimizer-level rewriting against a portable HIV SQL rewrite executed on the same engine under the same materialized view set.

7 EXPERIMENTAL SETUP

7.1 Datasets and Workloads

We perform the experiments with four workloads, namely JOB [36], SCALE [34], STATS [27, 39] and TPC-DS (SF=10) [17], with only JOB and SCALE sharing the same schema and dataset IMDB [36]. We choose them because they are widely used in the query optimization and view-based query rewriting literature [25, 26, 40, 53]. Additionally, these workloads cover a range of difficulty levels due to their significant diversities measured by join count (Table 11 in the supplementary material) and operator count in selection predicates (Table 12 in the supplementary material). JOB is the most complex one due to the highest join count and more logical connectives, null judgement, and string operations; STATS and TPC-DS have comparable join counts while the former involves more binary comparisons and type conversion, and the latter presents more arithmetic and range computations; SCALE is the simplest one with the lowest join count and fewest operator occurrences.

7.2 Platform and Implementation

Execution Engine and Cross Executability. We evaluate the following seven engines: PostgreSQL, Hive, Doris, StarRocks, Sys-A, Sys-B and Sys-C. For engines whose rewriting is plan-transparent (Doris, StarRocks, Sys-A, Sys-B, Sys-C), we compare two rewriters on the same engine: (i) the engine's native optimizer-level rewriting (measured via EXPLAIN and execution latency), and (ii) executing portable rewritten SQL produced by Hive. On PostgreSQL, we additionally include Calcite as a second SQL-transparent baseline. This instantiates the cross-platform protocol defined in Section 3.

Training for learned selectors. To support learned view selection baselines (UniView and GnnMV) and ensure a fair comparison, we split the original workload queries into training/validation/test sets with a 3:1:6 ratio, following prior work [25]. All methods are evaluated on the same test set. To construct training instances for learned selectors, we generate query–view pairs and label their utility as follows: i) use HIV to attempt a rewrite of a query with a candidate view; ii) execute both the original and rewritten queries on PostgreSQL and record their latencies; and iii) use the latency reduction as the utility label for the pair.

Storage Budget. The storage budget bounds the **total on-disk size of all selected materialized views** — not query-result caches or main-memory buffers. We measure view size using PostgreSQL's pg_total_relation_size, which includes the table heap and any indexes the selector would create. The primary budget is 1 GB; for sensitivity analysis we additionally sweep 0.1, 0.2, and 0.5 GB (the budgets are comparable to those used in prior work on MV selection [25, 26, 53] for the datasets). Given the underlying database sizes (IMDB 8.31 GB, used by JOB/SCALE; TPC-DS 21.96 GB), these budgets correspond to roughly 1.2–12% of IMDB and 0.5–4.5% of TPC-DS, spanning tight (<2% of the base data) through moderate (∼10%) regimes. This range overlaps the budgets used by BigSubs [31] (typically ≤2%) and UniView [53] (up to ∼10%), enabling direct comparison with prior work. The fact that 1 GB is moderate on IMDB but tight on TPC-DS is itself a finding (see the column-pruning case study in Section 10).

Hardware and Software. We run all non-commercial systems on an AWS EC2 instance. The instance model is c5a.16xlarge (64 vCPUs, 128 GB RAM, gp3 EBS and Ubuntu 24.04 OS). For Sys-A and Sys-B, the vendor-provided service stacks are also deployed on AWS.

7.3 Metrics

End-to-end metric. Since our evaluation considers a large number of enumerator–selector–rewriter combinations, we use a single end-to-end metric, *relative workload time saving*, to summarize the overall effectiveness of each combination. If a rewritten query is slower than its original counterpart, we treat it as *no rewrite* (i.e., we use the original query latency when aggregating workload time saving). All other metrics (including speedup vs regression) are reported at the stage level (enumeration, selection, and rewriting) to diagnose the sources of performance differences.

Unless otherwise stated, we evaluate each component using the best-performing combination of the other two components on each dataset within a fixed search space, e.g., when comparing rewriters, the input to each rewriter is the same set of views selected from a fixed combination of enumerator and selector.

Enumeration. We evaluate candidate enumerators using metrics for both scalability and candidate-set quality: (i) *Candidate set size* (number of generated view candidates); (ii) *Materialized view size distribution* (which affects downstream selection under budgets); (iii) *Candidate complexity*, including average join size (number of joined tables), the diversity of join orders, and the number/type of predicate operators in WHERE; (iv) *Per-view query coverage*, measured from the rewrite-relationship graph between views and queries: the distribution of how many queries a view can rewrite.

View selection. We evaluate selection methods under the same candidate set across different storage budgets, focusing on both effectiveness and overhead: (i) *Selection overhead*, including solver runtime for ILP-based methods and training/inference costs for

[2]https://github.com/StarRocks/starrocks/issues/11386

learned methods; (ii) *Filtering quality* (Section H in the supplementary material), i.e., how effectively a selector discards useless candidates. We set selection, rewriting and acceleration as three checkpoints and compare recommenders' behaviors.

Query rewriting. For rewriters, we report: (i) *Rewrite success rate* (fraction of queries for which a view is used, as indicated by the plan from `EXPLAIN` or by rewritten SQL when available); (ii) *Runtime impact*, including overall speedup distribution and the fraction of queries that improve versus regress.

8 END-TO-END AND CROSS-ENGINE PERFORMANCE (RQ1, RQ2)

We therefore report end-to-end results across all three stages under two complementary settings.

(i) PostgreSQL end-to-end. We evaluate the full modular pipeline on PostgreSQL, where candidate views, selected views, and rewritten SQL are observable and portable. This enables controlled comparisons across enumerators, selectors, and SQL-transparent rewriters by executing all rewritten queries on a common engine.

(ii) Cross-engine end-to-end. To incorporate systems that offer plan-transparent rewriting (i.e., exposed only through `EXPLAIN`), we execute the original workload on each engine. We then compare each engine's native optimizer-level rewriting against a portable SQL-rewrite baseline produced by Hive and execute the rewritten queries on the same engine under the same materialized-view set.

Figure 3 reports workload time saving for each pipeline on PostgreSQL with a fixed storage budget of 1 GB.

PostgreSQL end-to-end: no consistent winner and strong interaction effects. Across workloads, time savings vary substantially, and no single pipeline consistently dominates. For example, `ECSE–GnnMV–HIV` achieves the largest savings on STATS, whereas `UniView–BigSubs–HIV` performs best on JOB. Even within the same workload, changing only one stage can lead to large differences: on JOB, replacing `ECSE` with `Basic` in `ECSE–BigSubs–HIV` reduces time saving by 43.85 pp, whereas switching the selector from `BigSubs` to `GnnMV` under `Basic` increases savings by 47.53 pp, making it competitive with the best pipelines — so stage rankings must be read *jointly* with their upstream/downstream partners, not in isolation. We give the structural reason for this specific interaction (`BigSubs`'s creation-cost-dominated utility under-ranking the high-coverage views `ECSE` contributes; `GnnMV`'s learned cost recovering them under `Basic`) in Section 9.1, expanding on it with a detailed case study in Section 10.3, and the corresponding workload-feature→method-choice rule in the practical-guidance summary in Section 11. These results indicate strong interaction effects among enumeration, selection, and rewriting, supporting **RQ1** and motivating the stage-wise analyses in Sections 9.1–9.3.

Cross-engine end-to-end: performance is engine and workload dependent. We next evaluate cross-engine behavior using the best-performing modular pipelines identified on PostgreSQL under the cross-engine protocol. Across workloads, the best top-1 pipeline achieves 58–75% time saving (JOB: 64%, SCALE: 61%, STATS: 75%, TPC-DS: 58%), well above the corresponding per-engine median; the full per-engine top-1 breakdown is reported in the supplementary material. However, these top pipelines do not transfer uniformly across engines: the PostgreSQL top pipeline on JOB remains top-1 on Doris but yields smaller gains on StarRocks and limited gains on `Sys-A`, while the PostgreSQL top pipeline on SCALE improves on `Sys-A` but degrades on other engines. Overall, cross-engine MV benefits depend jointly on the selected view set, the rewriter's ability to exploit MVs, and engine-specific execution behavior.

When comparing portable `HIV` rewrites against each engine's native (optimizer-level) MV rewriting, we find that the best modular pipelines identified on PostgreSQL often yield larger time savings than the engine's native rewriter. In several cases the gap is substantial (e.g., JOB on Doris and `Sys-B`, and SCALE on `Sys-A`; Figure 4). On complex workloads such as JOB and STATS, the best modular pipelines achieve markedly higher time savings than the evaluated commercial systems in our setting (e.g., up to ∼60–70% vs. ∼15–20%; Figure 4). In contrast, on TPC-DS the gap narrows and, in some cases, commercial systems perform the best. Overall, these results indicate that MV-based query rewriting effectiveness is highly workload-dependent and engines exhibit different trade-offs across workloads (**RQ2**). We investigate the sources of these differences in the subsequent stage-wise analyses, including cases where candidate generation, view selection under budget, or rewrite applicability becomes the limiting factor.

Overall performance is determined by a long-tail minority of queries. We sort queries by original latency and plot the cumulative original latency and time savings achieved by the best combination (Figure 5). Each vertical dashed line distinguishes the top 5% slowest queries. Their contributions to (original latency, time saving) are: (36.93%, 51.42%) for JOB, (81.15%, 71.73%) for SCALE, (77.69%, 82.62%) for STATS and (39.13%, 12.97%) for TPC-DS. For those workloads where the best combinations are all non-commercial combinations (JOB, SCALE, STATS), the slowest queries not only contribute the most time saving, but their magnitude is also comparable to the original latency contribution. Only in TPC-DS, where the `Sys-B` pipeline is chosen, is the share of time saving by the slowest queries much smaller than the original latency contribution. That indicates that although `Sys-B` achieves the best performance on TPC-DS, it does not optimize the slowest queries very well in proportion to the rest of the queries.

9 STAGE-WISE ANALYSES

The end-to-end results exhibit strong interaction effects. We therefore perform controlled ablations: when analyzing one stage, we fix the other two stages to isolate its contribution.

9.1 Analysis of Enumerators

Enumerators differ in candidate count, coverage, and structure. The best enumerator varies by workload (`ECSE`/`UniView` on JOB/SCALE, `ECSE` on STATS, `Sys-B`[3] on TPC-DS), indicating that enumerators differ not just in how many candidates they generate but in the *types* of views they expose. We analyze candidate count, coverage, and structure to explain these differences.

Candidate-space and reuse potential. Table 3 reports candidate counts and the per-view covered-query ratio. `ECSE` and `Basic` enumerate far more candidates than `UniView`, and `Sys-B` the fewest; the `ECSE–Basic` gap widens on the most complex workload (JOB).

[3] `Sys-B` exposes views only after its internal selection.

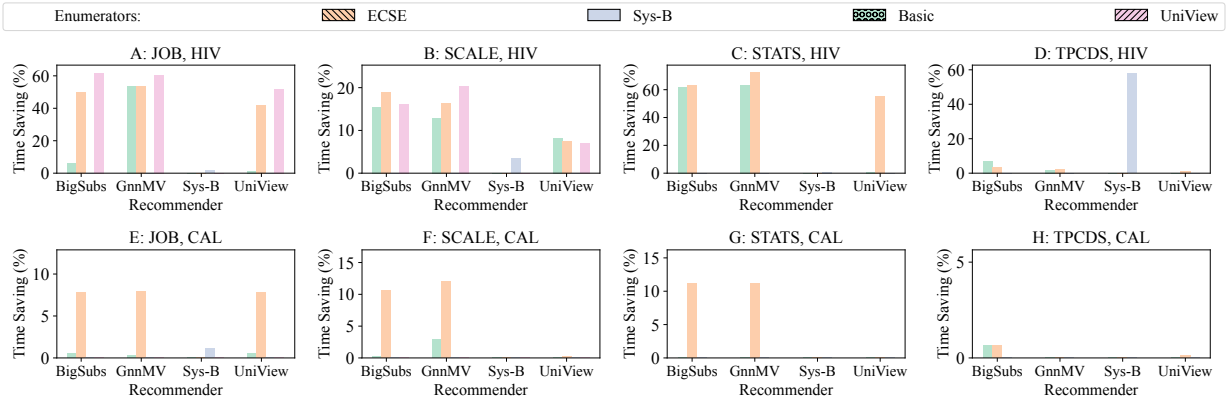


Figure 3: Workload time saving (%) of all (Enumerator, Selector, Rewriter) pipeline combinations on PostgreSQL.

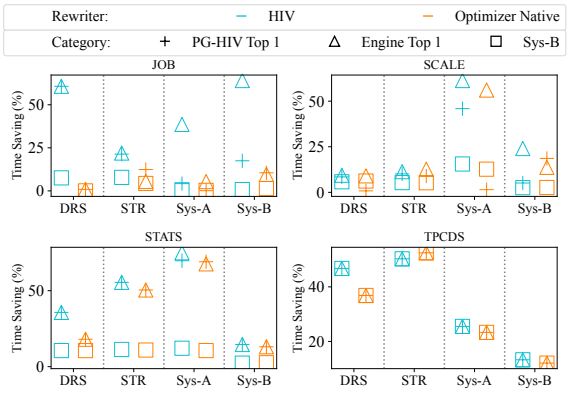


Figure 4: Cross-engine time saving under two rewrite modes. For each engine, three MV categories are examined to compare the performance of Optimizer Native (let the engine's native optimizer rewrite) against HIV (directly executing portable rewritten SQL generated by HIV). For MV categories, +: ranking top-1 under PG-HIV, △: ranking top-1 under this engine, □: Sys-B's auto MV selection. Budget=1GB.

Across workloads, candidate count does not directly imply higher reuse: on JOB, ECSE enumerates 373 candidates (vs. 146 for Basic) but has lower median and q95 coverage (q50: 9.09% vs. 18.18%; q95: 38.64% vs. 56.82%). In contrast, on STATS and TPC-DS, ECSE produces substantially higher tail coverage than Basic (e.g., max 47.23% vs. 13.49% on STATS; max 15.78% vs. 7.22% on TPC-DS), indicating that enumeration strategy can shift reuse from the "typical" candidate to a small set of "high-leverage" candidates. A consistent pattern across datasets is that coverage is skewed: most candidates cover few queries, while a small fraction achieve very high coverage (e.g., the max coverage reaches 70.45% on JOB). Whether downstream selection actually exploits this heavy-tailed structure depends on the selector's utility model: we show in Section 10.3 that BigSubs's creation-cost-dominated utility under-ranks exactly the high-coverage views in this tail on JOB, whereas GnnMV's learned cost prioritizes them — so the same enumerated candidate pool yields different downstream behavior depending on selector choice (the 47.53 pp difference under Basic reported in §8).

Table 3: Candidate-space statistics. Coverage is the fraction of workload queries covered by each candidate view. JO: number of unique join orders; J: mean number of joins; W: mean number of unique WHERE operators.

Workload	Enumerator	# Candidates	Coverage (%)				Complexity		
			Avg	q50	q95	Max	JO	J	W
JOB	Sys-B	11	3.31	4.55	4.55	4.55	5	5.27	0.64
	ECSE	373	14.84	9.09	38.64	70.45	243	2.34	0.06
	Basic	146	19.52	18.18	56.82	61.36	78	2.47	0.37
	UniView	95	3.78	2.27	13.64	15.91	43	2.13	7.98
SCALE	Sys-B	70	1.44	1.00	4.28	6.00	18	1.30	0.00
	ECSE	566	3.89	2.00	13.00	39.50	30	1.42	5.27
	Basic	569	3.38	1.50	11.50	39.50	56	1.41	5.41
	UniView	419	1.29	0.50	4.50	36.00	34	1.58	7.69
STATS	Sys-B	24	0.84	0.69	1.56	1.73	19	2.83	0.00
	ECSE	394	2.38	0.87	9.34	47.23	45	2.44	5.36
	Basic	392	1.34	0.69	3.88	13.49	32	2.50	5.80
TPC-DS	Sys-B	25	3.49	3.48	6.52	6.68	18	3.32	0.00
	ECSE	410	1.28	0.27	5.61	15.78	54	2.30	5.99
	Basic	305	0.77	0.27	3.96	7.22	19	2.44	7.23

Mechanism: structural bias, not candidate count. ECSE expands the space mainly through *join-structure variants*, producing 243 distinct join orders on JOB compared to Basic's 78 and UniView's 43

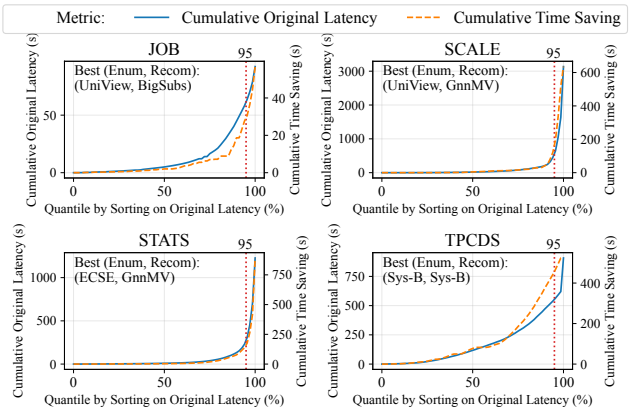


Figure 5: Cumulative Original Latency and Time Saving Distribution by Quantile. Queries are sorted by original latency in ascending order to determine the quantile.

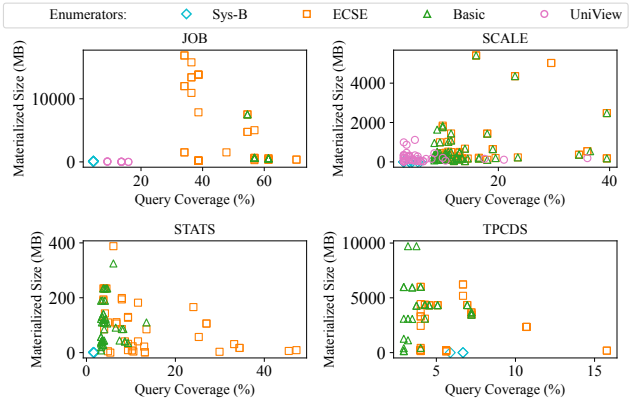


Figure 6: Query coverage vs. materialized size for the top 10% of views by query coverage from each enumerator. Size measured on PostgreSQL.

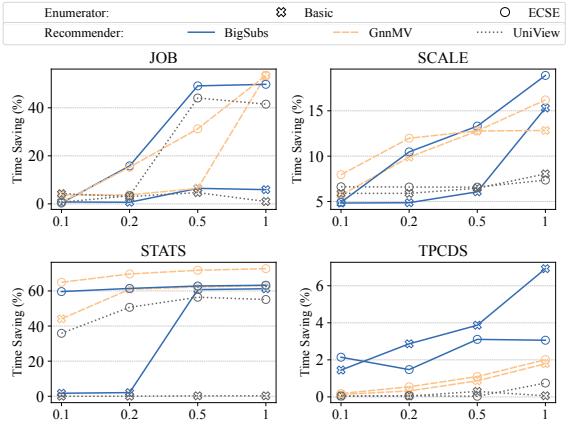


Figure 7: Time Saving of Each Enumerator-Recommender Combination under Different Storage Budgets

at comparable join counts (see Table 18 in the supplementary material), leading to high-coverage candidates in the tail. Enumerators also differ in predicate richness (`UniView` ~8 `WHERE` operators per view vs. `Sys-B`'s ~0), so the same selector and rewriter can yield very different end-to-end savings across enumerators.

Candidate structural characteristics. Table 18 (in the supplementary material) summarizes structural properties of the enumerated candidates. On JOB, `ECSE` generates far more distinct join orders than other enumerators (243 vs. 78 for `Basic` and 43 for `UniView`), while the average join count is comparable (2.34 vs. 2.47/2.13). Combined with the coverage per view, this indicates that `ECSE` expands the candidate space primarily through *structural variants* rather than enumerating larger join subexpressions, and helps explain why `ECSE` contains higher-*tail*-coverage candidates on JOB.

Coverage vs. size: high reuse may not imply large views. A common intuition is that high-coverage views must be more general (fewer predicates, more projected columns) and therefore larger in size. Figure 6 tests this by plotting the materialized size distribution of the top 10% of candidates ranked by coverage for each enumerator (Table 3). We find that the size–coverage trade-off is not universal: on JOB, STATS, and TPC-DS, `ECSE` produces many high-coverage candidates that are also relatively small, whereas other enumerators more closely follow the expected trade-off. On SCALE, where query structure is simpler, `ECSE` behaves similarly to `Basic`. Overall, these results suggest that for complex workloads, `ECSE` can expose more space-effective candidates (high reuse with modest materialization size), which helps downstream selection under budget.

Takeaways. (RQ1/RQ3) Enumeration sets the achievable upper bound on end-to-end savings — a *cap*, not a *cause*. Across the end-to-end pipeline combinations evaluated in Section 8 (Figure 3), no downstream selector or rewriter recovers the savings lost when the enumerator omits a high-coverage candidate: e.g., `Basic`'s top-1 candidate on JOB covers 18.18% of queries vs. `ECSE`'s 70.45%, and no selector–rewriter pair on the `Basic` candidate set matches the `ECSE`-best pipeline's end-to-end time saving on JOB.

(RQ3) Candidate count is not a proxy for usefulness: larger candidate spaces do not necessarily yield higher reuse or savings; reuse (coverage) is typically *heavy-tailed*, driven by a small set of high-impact views.

(RQ3) Structural bias matters: enumerators differ in join-order exploration and predicate specialization, which shifts where reuse appears and affects which queries downstream stages can accelerate.

9.2 Analysis of Selectors

Selectors differ more in robustness than in average performance. In the end-to-end results (Figure 3), no selector wins universally: the ranking of `BigSubs`, `GnnMV`, and `UniView` changes across workloads and (enumerator, rewriter) settings. We therefore study selector robustness through a storage-budget sweep over the three budget-aware selectors, excluding `Sys-B` because its integrated recommender does not expose an explicit storage-budget interface.

Budget sensitivity and interaction with enumerators. Figure 7 varies the storage budget (0.1/0.2/0.5/1 GB) under two enumerators (`ECSE` and `Basic`), while fixing the rewriter to `HIV` and the engine to `PG`. Overall, selector behavior is highly workload-dependent. On JOB, SCALE, and STATS, `GnnMV` is usually best or near-best across budgets, while `BigSubs` is competitive but occasionally yields limited savings; `UniView` generally underperforms. On TPC-DS, `BigSubs` performs best, but the absolute savings are smaller, indicating fewer exploitable MV opportunities under this budget range.

Two patterns explain why the selector choice matters. First, savings often show threshold effects: moving from 0.2 to 0.5 GB or from 0.5 to 1 GB can produce abrupt gains, suggesting that a few high-impact views sit near budget boundaries. Second, selectors interact strongly with enumerators. For example, on JOB, `Basic`-based pipelines perform poorly with `BigSubs` and `UniView`, but become competitive when paired with `GnnMV` at 1 GB. Thus, a selector can sometimes compensate for a weaker candidate set by prioritizing the remaining high-impact views.

Diagnosing cases with large selector gaps. To understand when selector matters most, we drill down into the outlier where `GnnMV`

substantially outperforms BigSubs under a tight budget. Since both methods saturate the storage budget and are cost-aware, the gap cannot be explained by budget under-utilization; instead, it reflects differences in *which* views are prioritized. We find evidence that BigSubs can under-rank certain high-impact views due to its simplified utility model, leading to suboptimal selections when benefits depend on factors that are not well captured by the estimator (§10.3).

Latency overhead. Selection overhead is consistent across workloads: BigSubs is cheapest (no training; sub-second ILP), GnnMV adds 288–1,551 s of training, and UniView is the most expensive; the integrated Sys-B scales more gently with workload size (2.27× from JOB to STATS vs. ∼5.3–5.5× for the learned selectors). Full numbers are in the supplementary material.

Takeaways. (RQ1) Learned selectors yield more stable performance across budgets on complex workloads, with moderate training/inference overhead, compared to ILP-based BigSubs.
(RQ3) Budget sensitivity is significant: selector rankings can flip between 0.1–0.2 GB and 0.5–1 GB, and savings often exhibit threshold effects when a small set of high-impact views becomes feasible.
(RQ3) Under tight budgets, performance is driven primarily by *view ranking* rather than budget utilization; a cost-based utility model can under-rank a few high-impact views.

9.3 Analysis of Rewriters

Rewriter capability often caps end-to-end time savings. Regarding **RQ3**, we find that the rewriter frequently determines how much benefit upstream enumeration and selection can translate into actual workload savings. With the enumerator and selector fixed, replacing the rewriter can change the time saving substantially. For example, on JOB with the same selected view set (Basic + GnnMV), switching from HIV to Sys-A's native rewriter reduces workload time saving from ∼40% to ∼5% (Figure 4). This suggests that, in many scenarios, the limiting factor is not cost estimation (since HIV does not require physical materialization to produce a rewrite) or view ranking, but whether the rewriter can *recognize* applicable views and *apply* effective substitutions. We further analyze rewrite success, view usage, and representative failure modes below.

Rewrite success rate varies across engines and workloads. Table 4 reports rewrite success rates under each rewriter, grouped by workload and enumerator.[4] Across JOB, STATS, and TPC-DS, HIV is the most consistently effective rewriter, achieving the highest success rates in 9 out of 14 workload–enumerator settings; STR and DRS lead in fewer cases. Among commercial systems, Sys-A more frequently exhibits higher rewrite-success rates than Sys-B in these workloads. Sys-C exhibits the most variable behavior across workloads: only 1–2% rewrite success on JOB (limited support for the workload's complex multi-join patterns), 4–15% on SCALE and STATS, but 42–78% on TPC-DS. In particular, it cannot handle aggregate-heavy templates enumerated by Sys-B well (0% in SCALE, STATS, and TPC-DS). However, there is no universal winner: in some settings (e.g., parts of STATS and TPC-DS), multiple systems reach near-100% success rates, indicating that these workloads contain patterns well covered by several optimizers. In contrast, on SCALE the ranking changes: STR becomes the top-performing open-source rewriter and Sys-B becomes the top-performing commercial rewriter, with comparable success rates.

Overall, these results reinforce that rewriter behavior is strongly workload- and engine-dependent, and can be a dominant source of end-to-end variability even when the view set is fixed. We conduct a case study in Section F (the supplementary material) to investigate why HIV achieves higher success rates in most settings, and compare HIV with DRS in depth on STATS. Additionally, via a case study in Section G in the supplementary material, we show that MVs can also be "forced" into rewrites on certain engines, but such forced usage does not improve performance, and such risky rewrites can sometimes be easily detected.

Runtime impact varies substantially, revealing different robustness profiles. Figure 8 presents the rewrite impact on each query on JOB and SCALE. Each vertical bar corresponds to a successfully rewritten query, and its height is the relative speedup (positive) or slowdown (negative) compared to executing the original query on the same engine. The distributions expose substantial heterogeneity within a single workload: (i) many successful rewrites yield large gains, but a non-trivial fraction produce regressions, showing that rewrite "success" does not imply performance improvement; (ii) many rewrites deliver limited or even zero benefit, e.g., HIV and STR on JOB, HIV and DRS on SCALE; (iii) rewriters differ not only in the fraction of positive pairs, but also in tail behavior, e.g., some exhibit both heavy negative and positive tails (severe regressions as well as large improvements, e.g., HIV and STR

Table 4: Rewrite success rate (%) and count of successful rewrites for each rewriter. Green and yellow shading mark the highest success rate among non-commercial and commercial rewriters, respectively. Bold entries denote the overall highest success rate across all rewriters.

WL	Enum.	CAL	Sys-A	Sys-B	Sys-C	DRS	HIV	STR
JOB	UniView	4 (2%)	31 (15%)	146 (71%)	2 (1%)	7 (3%)	**152 (73%)**	146 (71%)
	ECSE	4 (7%)	**49 (80%)**	15 (25%)	1 (2%)	0 (0%)	39 (64%)	26 (43%)
	Sys-B	5 (26%)	16 (84%)	16 (84%)	4 (21%)	0 (0%)	**19 (100%)**	16 (84%)
	Basic	6 (10%)	**50 (79%)**	11 (17%)	5 (8%)	2 (3%)	**50 (79%)**	14 (22%)
SCALE	UniView	0 (0%)	15 (3%)	**404 (87%)**	45 (10%)	23 (5%)	186 (40%)	**404 (87%)**
	ECSE	51 (4%)	47 (4%)	1231 (94%)	49 (4%)	51 (4%)	78 (6%)	**1243 (95%)**
	Sys-B	0 (0%)	**281 (100%)**	**281 (100%)**	0 (0%)	**281 (100%)**	**281 (100%)**	**281 (100%)**
	Basic	76 (5%)	116 (8%)	1413 (93%)	63 (4%)	104 (7%)	152 (10%)	**1420 (93%)**
STATS	ECSE	110 (3%)	2552 (78%)	2237 (69%)	260 (8%)	747 (23%)	**3200 (98%)**	2214 (68%)
	Sys-B	0 (0%)	**230 (100%)**	**230 (100%)**	0 (0%)	224 (97%)	**230 (100%)**	**230 (100%)**
	Basic	54 (5%)	661 (66%)	788 (79%)	145 (15%)	389 (39%)	**887 (89%)**	775 (78%)
TPCDS	ECSE	14 (12%)	97 (83%)	61 (52%)	49 (42%)	**114 (97%)**	103 (88%)	61 (52%)
	Sys-B	0 (0%)	**469 (100%)**	**469 (100%)**	0 (0%)	303 (65%)	**469 (100%)**	**469 (100%)**
	Basic	14 (16%)	71 (83%)	58 (67%)	67 (78%)	59 (69%)	**86 (100%)**	58 (67%)

[4]We measure success rate as the fraction of query–view pairs for which the rewriter produces a query/plan that references an MV, under a fixed pool of MVs.

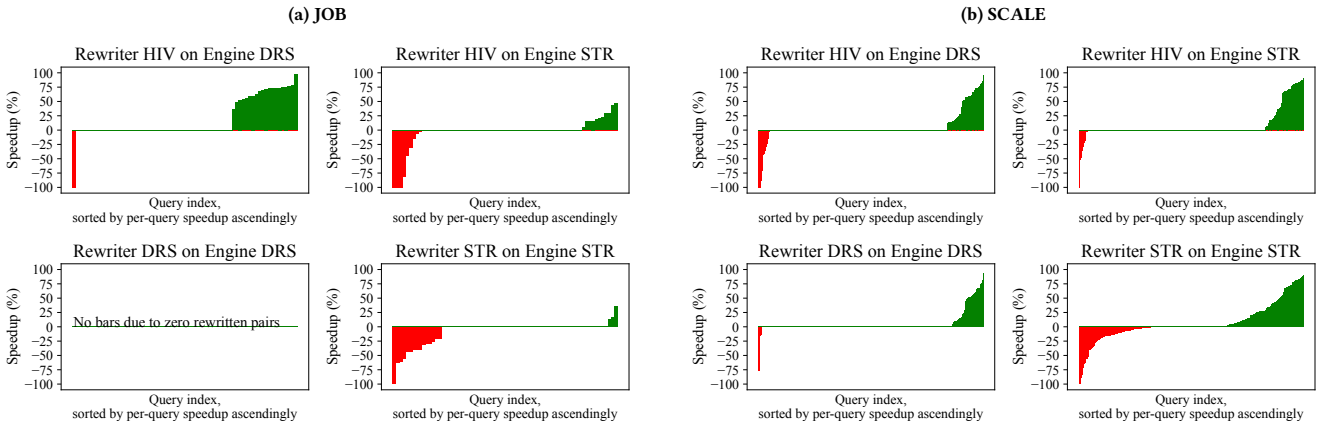


Figure 8: Per-query runtime impact of each rewriter on JOB and SCALE. Each bar is one successfully rewritten query; bar height is the relative speedup. Positive (accelerated) and negative (slowdown) speedups are marked in green and red respectively. Queries on the X-axis are sorted left-to-right by their speedup magnitudes; the ordering is independent per subplot.

for SCALE), whereas others exhibit non-symmetric tails (e.g., `HIV` on Doris engine for JOB, `STR` for JOB).

Takeaways. (RQ2) Rewriter behavior is *engine- and workload-dependent*, with no universal winner. Under a fixed view set (Table 4), `HIV` achieves the highest rewrite-success rate in 9 of 14 workload–enumerator settings across JOB/STATS/TPC-DS, yet `STR` leads on SCALE and several systems tie near 100% elsewhere, supporting the finding that auto-MV rewrite does not imply consistent rewrite usage.
(RQ1/RQ2) Rewrite *success* is decoupled from *benefit*, and robustness lives in the *tails*. Under data-distribution drift, `HIV`'s rewrite-success rate stays nearly flat (89% → 91%) while `DRS`'s collapses (99% → 28%) on the same `ECSE`-enumerated view set (Table 23 in the supplementary material); and on JOB, switching only the rewriter (`HIV`→`Sys-A`) under a fixed view set cuts time saving ~40%→~5% (Figure 4). Rare but severe negative-speedup tails on specific (rewriter, engine) pairs can therefore dominate workload-level outcomes.
(RQ2) Under the cross-engine protocol, `HIV`-rewritten SQL provides a controlled baseline against native optimizer-level rewriting, and its advantage is workload-dependent: `HIV` dominates plan-transparent rewriters on STATS-`ECSE` (98% vs. 23–78%) but trails on TPC-DS-`ECSE` (88% vs. `DRS` 97%) (Table 4)—most useful where engine-native rewriting is weakest.

9.4 Robustness Analysis

The preceding analyses use a controlled steady-state setting: the MV-construction workload matches the evaluation workload, the database instance is fixed, and the same resource environment is used throughout. We add three robustness checks beyond this: workload drift, data-distribution drift, and memory pressure. Table 5 summarizes the main results. We elaborate on the workload-drift analysis here because it directly tests the representativeness assumption of offline MV construction; full data-skew and hardware-pressure case studies are provided in the supplementary material.

9.4.1 Workload Drift. The workload-drift experiment tests whether the workload used for MV construction is representative of the workload used for evaluation. We study this using two workloads (JOB and SCALE) with the rewriter fixed to `HIV`, the execution engine to `PG`, and the storage budget to 1 GB. We report relative workload time saving and average results across selectors (Table 6). The shrink factor is computed as the matched-setting saving divided by the mismatched-setting saving; larger values indicate stronger degradation under drift.

Two patterns stand out. First, `ECSE` is substantially more robust to workload drift than `UniView`: its shrink factors stay near 1 (1.38 and 0.73), whereas `UniView` degrades sharply (3.18 and +∞). The

Table 5: Summary of robustness experiments. Details are in the supplementary material.

Stress	What changes	Key result	Takeaway
Workload drift (§9.4.1)	MV construction workload differs from evaluation workload on IMDB.	`ECSE` is stable (shrink factors 1.38/0.73), while `UniView` is sensitive (3.18/+∞).	Join-graph candidates transfer better than predicate-specialized candidates.
Data-distribution drift (§9.4.2 and §E.1.1 in the supplementary material)	Same TPC-DS templates; balanced TPC-DS vs. skewed DSB instance.	On skew-sensitive `PG` queries, `GnnMV` drops 71.67%→49.71%, while `BigSubs` drops 61.04%→53.38%.	Skew affects both realized speedup and selector robustness.
Hardware / memory pressure	MV recommendation and evaluation run across abundant vs. constrained EC2 settings.	Recommended MV sets often remain useful across environments, but native rewrite success can change for some engines.	MV selection transfers reasonably across resources; rewriting remains engine-sensitive.

Table 6: Time saving (%) under workload drift. Shrink factor is computed as matched / mismatched.

Enumerator	Eval. workload	Construction workload			
		JOB	SCALE	Shrink	JOB + SCALE
`UniView`	JOB	20.02	6.30	3.18	23.69
	SCALE	0.00	13.32	+∞	6.39
`ECSE`	JOB	49.21	35.63	1.38	46.63
	SCALE	13.47	9.86	0.73	10.73

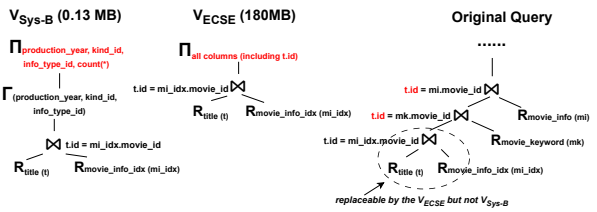


Figure 9: Rewriting Failure Case of Sys-B.

likely reason is that ECSE derives candidates from workload-level join graphs, which can transfer across related query mixes, while UniView produces more predicate-specialized candidates that may fail to match shifted workloads. Second, cross-workload construction can sometimes outperform in-distribution construction: for ECSE on SCALE evaluation, constructing from JOB yields 13.47% time saving, higher than 9.86% from SCALE construction. This suggests that a richer historical workload can expose reusable join structures that benefit a simpler future workload.

9.4.2 Data-Distribution Skewness and Hardware Resource Pressure. The other two robustness checks affect different stages. Under DSB skew, realized time saving and native rewrite success both vary substantially across engines, indicating that skew affects not only original-query runtime but also optimizer-level MV usage. Under memory pressure, selected MV sets often remain useful across environments, but native rewrite success can still change, reinforcing that rewriting remains engine-sensitive. Detailed evidence is reported in the supplementary material (§E.1.1).

10 CASE STUDY

10.1 Case 1: Comparing Non-commercial vs. Sys-B Enumerator Across Workloads

From the results in Figure 3 and Figure 4, we observe that Sys-B exhibits large time savings and high query coverage on TPC-DS as an enumerator. A natural question is why these benefits are substantial on TPC-DS but do not consistently transfer to the other workloads. A key design choice of Sys-B is aggressive column pruning when generating view candidates, producing substantially lighter-weight materialized views (Section 4 and Figure 2). This strategy is particularly effective on TPC-DS. For example, the 25 views enumerated and recommended by Sys-B occupy only 30 MB of storage space in total, whereas a comparable view generated by ECSE — sharing the same join pattern but without column pruning — takes 2.3 GB. Under a space budget of up to 1 GB, Sys-B therefore provides many more feasible candidates.

However, the same column pruning can also lead to more rewrite failures and, consequently, weaker performance on other workloads than column-pruning-free enumerators. To illustrate, Figure 14 (in the supplementary material) and Figure 9 show an original query from workload SCALE and two selected views: one enumerated by ECSE (V_{ECSE}) and the other by Sys-B ($V_{\text{Sys-B}}$). While the two views share the same join over two relations, V_{ECSE} retains all columns, whereas $V_{\text{Sys-B}}$ keeps only three and omits `title.id`.

Table 7: Budget utilization rate and time saving achieved by each selector on STATS + Basic at the 102 MB budget. Rewriter = HIV, engine = PG.

Selector	MV count	Utilization (%)	Time saving (%)
BigSubs	23	100.00	1.71
GnnMV	58	99.97	44.11
UniView	1	3.43	0.02

In the original query, `title.id` is required for joining additional relations; as a result, only V_{ECSE} can be used for rewriting.

This explains why Sys-B appears particularly strong on TPC-DS. Its database is substantially larger, so a fixed 1 GB budget is relatively tight compared to the database scale. Under such constraints, pruning-free enumerators are more likely to generate candidates that exceed the budget, even if they provide better coverage in general. In contrast, Sys-B's pruning yields many budget-feasible candidates and thus performs well. For the other workloads with smaller scale, the same 1 GB budget is effectively more generous: many pruning-free candidates remain feasible, and the dominant limitation shifts from storage to query coverage. In short, no single enumerator dominates across all workloads, which motivates evaluating multiple enumerators under different budget settings.

10.2 Case 2: Diagnosing Rewrite Gaps Between HIV and DRS

We diagnose the rewrite gap between HIV (3,200 pairs) and DRS (747 pairs) on STATS + ECSE, which shows two structural advantages of HIV. First, join-matching flexibility: DRS rewrites a view only when the join graph matches a small set of pre-recognized patterns; an additional joined relation breaks the match. Second, predicate pushdown: when a query carries additional selection predicates beyond the view definition, HIV applies them atop the view scan, while DRS rejects the rewrite due to stricter predicate-compatibility enforcement. More details are provided in Section F in the supplementary material.

10.3 Case 3: BigSubs Under-ranks High-Coverage Views (STATS, 102 MB)

To make the BigSubs ranking-failure mechanism (introduced in Section 9) concrete, we present the smallest-budget instance of the gap. Setup: workload STATS, enumerator Basic, rewriter HIV, engine PG, and a tight 102 MB budget. Both BigSubs and GnnMV are pure-ILP selectors that consume the same candidate pool; the question is which views each selects.

Headline gap. Under this configuration, BigSubs achieves 1.71 % workload time saving while GnnMV achieves 44.11 % — a 42.4 pp gap. Both selectors saturate the budget (BigSubs 23 views at 100 % utilization; GnnMV 58 views at 99.97 %; Table 7), so the gap *cannot* be explained by budget under-utilization. The disagreement is in *which* views the two selectors pick.

Where the rankings disagree. Table 8 shows the three highest-utility views chosen by BigSubs (top half) and by GnnMV (bottom half). Two patterns are immediate. First, BigSubs's top three views (MV 121, 263, 7340) collectively contribute essentially zero workload time saving — 0.00, 0.03, 0.00 % respectively — yet BigSubs ranks

Table 8: Top-three views ranked by BigSubs versus top-three ranked by GnnMV on STATS + Basic + 102 MB. *Estimated benefit* is each selector's utility score; *actual contributed time saving* is measured post-rewrite on PG.

MV id	Est. benefit (BigSubs)	Est. benefit (GnnMV)	Mat. size (MB)	Eventually selected by	Actual contributed TS (%)
121	4.64	6.15	44.50		0.00
263	4.15	0.35	41.00	Only BigSubs	0.03
7340	3.03	0	10.00		0.00
1017	2.9	73.05	35.50		8.94
3232	1.35	218.77	36.50	Only GnnMV	19.26
906	1.04	40.44	10.50		12.55

Table 9: BigSubs utility breakdown for the relevant views on STATS + Basic + 102 MB. Recall $u = (a - b) \times$ *count*, where a is creation cost and b is sequential scanning cost.

MV id	Creation cost (a)	Scan cost (b)	Per-view utility ($a - b$)	Covered query count	Estimated utility ($cnt \cdot (a-b)$)
7340	3.11	0.08	3.03	1	3.03
3232	0.40	0.29	0.11	12	1.35
1017	0.36	0.29	0.07	39	2.90
906	0.12	0.08	0.04	28	1.04

them highest because their utility numbers (4.64, 4.15, 3.03) are the largest in its model. Second, the three views that account for the bulk of GnnMV's 44.11 % time saving (MV ids 1017, 3232, 906; actual contributed 8.94, 19.26, 12.55 %) appear far down BigSubs's utility ranking (estimated benefit 2.9, 1.35, 1.04). BigSubs *has these views in its candidate pool* but ranks them too low to be selected within the budget.

Mechanism. BigSubs's utility is computed as $u = (creation_cost - scan_cost) \times count$. Table 9 decomposes this for the relevant views: MV 7340 picked by BigSubs has high per-view utility ($u = 3.03$) because its creation cost (3.11) is large, but it covers only 1 query. GnnMV's actually helpful picks (MV 1017, 3232, 906) cover many queries (39, 12, 28) but their per-view utility ($a - b = 0.07, 0.11, 0.04$) is tiny because their creation cost is modest. Multiplying by *count* cannot lift them above BigSubs's creation-cost-dominated picks, so the moderate-cost / high-coverage views never enter BigSubs's top set. This is the same mechanism that explains the 47.53 pp BigSubs/GnnMV gap on JOB +1 GB reported in Section 8; the STATS + 102 MB instance shown here is simply the most extreme case in our budget sweep. The takeaway is that BigSubs-style cost-only utility models systematically penalize the precise heavy-tail views (moderate creation cost, high query coverage) that ECSE-class enumeration is best at producing.

11 DISCUSSION

Our evaluation shows that MV-based rewriting remains workload- and engine-dependent: end-to-end gains are governed by interactions among candidate enumeration, budget-constrained selection, and rewrite applicability. We therefore summarize the main diagnostic signals for practitioners in Table 10; the goal is not to prescribe a universal pipeline, but to identify which stage is likely to be limiting under observable workload and engine properties.

Practical Guidance. Drawing only on evidence already presented above, we summarize the cross-stage findings in a compact decision table (Table 10) and a metrics-to-watch checklist for practitioners building MV-based query-rewriting pipelines.

Table 10: Workload-feature → method-choice decision suggestion, with paper-side evidence pointers.

Workload feature	Symptom / metric	Recommended action	Evidence
Join-heavy queries; heavy-tailed view coverage	Top-1 view coverage substantially above the median (e.g., >30 % on JOB/STATS)	Use cross-query-pattern enumerators such as ECSE; plan-confined enumerators may miss high-leverage views	Table 3; Fig. 6
Tight budget vs. large base data	Budget-to-DB ratio is small; useful candidates exceed budget	Prefer compact or column-pruned candidates; inspect coverage-vs-size trade-off	Case study §10.1; Table 17
High predicate / type-conversion diversity	Predicate structure is diverse (STATS-like profile)	Validate selector ranking carefully; learned selectors such as GnnMV may help.	Table 12; selector analysis in §9
Plan-transparent engine, or low rewrite-success	Native rewrite-success is much lower than portable HIV on the same MV set	Validate with a portable HIV rewrite baseline; inspect the negative-speedup tail before deploying.	Table 4; Fig. 4; Table 23

Key Metrics. Users should monitor three classes of diagnostics: candidate reuse potential, budget sensitivity, and rewrite risk. Candidate reuse (enumeration) is captured by view coverage and coverage-vs-size; budget sensitivity (selection) is captured by time-saving curves under different storage budgets; and rewrite risk (rewriting) is captured by MV usage rate and the negative-speedup tail.

Limitations and Future Work. First, cross-engine comparison cannot fully eliminate system-specific effects. Even though our protocol has controlled multiple factors, some optimizer decisions still remain opaque in plan-transparent systems. Our results should therefore be interpreted as a comparison of practical MV-exploitation behavior, not as a fully isolated comparison of rewrite algorithms. Second, we evaluate MVs statically. We do not model MV refresh cost, update frequency, or maintenance overhead. Incorporating maintenance-aware objectives and richer optimizer observability are important directions for extending this benchmark. We leave these valuable topics for future work.

12 CONCLUSION

We present an end-to-end, cross-engine evaluation of MV-based query rewriting across candidate enumeration, view selection, and query rewriting. We show that pipeline components cannot be ranked independently: a method that appears best under one fixed context may not be best once the candidate space, selector, rewriter, and engine change together. Making MV-based rewriting predictable in practice will therefore require co-designing high-leverage candidate generation, budget-robust selection, and rewrite applicability with explicit regression control. Our modular framework and cross-engine protocol provide a basis for reproducible comparison.

ACKNOWLEDGMENTS

We acknowledge the support of the Natural Sciences and Engineering Research Council of Canada (NSERC).

REFERENCES

[1] 2025. Apache Doris. https://doris.apache.org/.

[2] 2025. Apache Hive. https://hive.apache.org/.

[3] 2025. AWS Redshift. https://aws.amazon.com/redshift/.

[4] 2025. Google Cloud BigQuery. https://cloud.google.com/bigquery?hl=en.

[5] 2025. Oracle Database. https://www.oracle.com/ca-en/database/.

[6] 2025. Snowflake. https://www.snowflake.com/en/.

[7] 2025. StarRocks. https://www.starrocks.io/.

[8] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of Databases*. Addison-Wesley. http://webdam.inria.fr/Alice/

[9] Sanjay Agrawal, Surajit Chaudhuri, and Vivek R. Narasayya. 2000. Automated Selection of Materialized Views and Indexes in SQL Databases. In *VLDB 2000, Proceedings of 26th International Conference on Very Large Data Bases, September 10-14, 2000, Cairo, Egypt*, Amr El Abbadi, Michael L. Brodie, Sharma Chakravarthy, Umeshwar Dayal, Nabil Kamel, Gunter Schlageter, and Kyu-Young Whang (Eds.). Morgan Kaufmann, 496–505. http://www.vldb.org/conf/2000/P496.pdf

[10] Rafi Ahmed, Randall G. Bello, Andrew Witkowski, and Praveen Kumar. 2020. Automated Generation of Materialized Views in Oracle. *Proc. VLDB Endow.* 13, 12 (2020), 3046–3058. https://doi.org/10.14778/3415478.3415533

[11] Edmon Begoli, Jesús Camacho-Rodríguez, Julian Hyde, Michael J. Mior, and Daniel Lemire. 2018. Apache Calcite: A Foundational Framework for Optimized Query Processing Over Heterogeneous Data Sources. In *Proceedings of the 2018 International Conference on Management of Data, SIGMOD Conference 2018, Houston, TX, USA, June 10-15, 2018*, Gautam Das, Christopher M. Jermaine, and Philip A. Bernstein (Eds.). ACM, 221–230. https://doi.org/10.1145/3183713.3190662

[12] Randall G. Bello, Karl Dias, Alan Downing, James J. Feenan Jr., James L. Finnerty, William D. Norcott, Harry Sun, Andrew Witkowski, and Mohamed Ziauddin. 1998. Materialized Views in Oracle. In *VLDB'98, Proceedings of 24rd International Conference on Very Large Data Bases, August 24-27, 1998, New York City, New York, USA*, Ashish Gupta, Oded Shmueli, and Jennifer Widom (Eds.). Morgan Kaufmann, 659–664. http://www.vldb.org/conf/1998/p659.pdf

[13] Ashok K. Chandra and Philip M. Merlin. 1977. Optimal Implementation of Conjunctive Queries in Relational Data Bases. In *Proceedings of the 9th Annual ACM Symposium on Theory of Computing, May 4-6, 1977, Boulder, Colorado, USA*, John E. Hopcroft, Emily P. Friedman, and Michael A. Harrison (Eds.). ACM, 77–90. https://doi.org/10.1145/800105.803397

[14] Surajit Chaudhuri, Ravi Krishnamurthy, Spyros Potamianos, and Kyuseok Shim. 1995. Optimizing Queries with Materialized Views. In *Proceedings of the Eleventh International Conference on Data Engineering, March 6-10, 1995, Taipei, Taiwan*, Philip S. Yu and Arbee L. P. Chen (Eds.). IEEE Computer Society, 190–200. https://doi.org/10.1109/ICDE.1995.380392

[15] Leonardo Weiss Ferreira Chaves, Erik Buchmann, Fabian Hueske, and Klemens Böhm. 2009. Towards materialized view selection for distributed databases. In *EDBT 2009, 12th International Conference on Extending Database Technology, Saint Petersburg, Russia, March 24-26, 2009, Proceedings (ACM International Conference Proceeding Series)*, Martin L. Kersten, Boris Novikov, Jens Teubner, Vladimir Polutin, and Stefan Manegold (Eds.), Vol. 360. ACM, 1088–1099. https://doi.org/10.1145/1516360.1516484

[16] Sara Cohen, Werner Nutt, and Yehoshua Sagiv. 2006. Rewriting queries with arbitrary aggregation functions using views. *ACM Trans. Database Syst.* 31, 2 (2006), 672–715. https://doi.org/10.1145/1138394.1138400

[17] Transaction Processing Performance Council(TPC). 2025. TPC-DS Vesion 2 and Version 3. *http://www.tpc.org/tpcds/* (2025).

[18] Georgios Giannikis, Darko Makreshanski, Gustavo Alonso, and Donald Kossmann. 2014. Shared Workload Optimization. *Proc. VLDB Endow.* 7, 6 (2014), 429–440. https://doi.org/10.14778/2732279.2732280

[19] Jonathan Goldstein and Per-Åke Larson. 2001. Optimizing Queries Using Materialized Views: A practical, scalable solution. In *Proceedings of the 2001 ACM SIGMOD international conference on Management of data, Santa Barbara, CA, USA, May 21-24, 2001*, Sharad Mehrotra and Timos K. Sellis (Eds.). ACM, 331–342. https://doi.org/10.1145/375663.375706

[20] Stéphane Grumbach, Maurizio Rafanelli, and Leonardo Tininini. 1999. Querying Aggregate Data. In *Proceedings of the Eighteenth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems, May 31 - June 2, 1999, Philadelphia, Pennsylvania, USA*, Victor Vianu and Christos H. Papadimitriou (Eds.). ACM Press, 174–184. https://doi.org/10.1145/303976.303994

[21] Ashish Gupta, Venky Harinarayan, and Dallan Quass. 1995. Aggregate-Query Processing in Data Warehousing Environments. In *VLDB'95, Proceedings of 21th International Conference on Very Large Data Bases, September 11-15, 1995, Zurich, Switzerland*, Umeshwar Dayal, Peter M. D. Gray, and Shojiro Nishio (Eds.). Morgan Kaufmann, 358–369. http://www.vldb.org/conf/1995/P358.PDF

[22] Himanshu Gupta. 1997. Selection of Views to Materialize in a Data Warehouse. In *Database Theory - ICDT '97, 6th International Conference, Delphi, Greece, January 8-10, 1997, Proceedings (Lecture Notes in Computer Science)*, Foto N. Afrati and Phokion G. Kolaitis (Eds.), Vol. 1186. Springer, 98–112. https://doi.org/10.1007/3-540-62222-5_39

[23] Himanshu Gupta and Inderpal Singh Mumick. 2005. Selection of Views to Materialize in a Data Warehouse. *IEEE Trans. Knowl. Data Eng.* 17, 1 (2005), 24–43. https://doi.org/10.1109/TKDE.2005.16

[24] Alon Y. Halevy. 2001. Answering queries using views: A survey. *VLDB J.* 10, 4 (2001), 270–294. https://doi.org/10.1007/S007780100054

[25] Yue Han, Chengliang Chai, Jiabin Liu, Guoliang Li, Chuangxian Wei, and Chaoqun Zhan. 2022. Dynamic Materialized View Management using Graph Neural Network. https://api.semanticscholar.org/CorpusID:253519233

[26] Yue Han, Guoliang Li, Haitao Yuan, and Ji Sun. 2021. An Autonomous Materialized View Management System with Deep Reinforcement Learning. In *37th IEEE International Conference on Data Engineering, ICDE 2021, Chania, Greece, April 19-22, 2021*. IEEE, 2159–2164. https://doi.org/10.1109/ICDE51399.2021.00217

[27] Yuxing Han, Ziniu Wu, Peizhi Wu, Rong Zhu, Jingyi Yang, Liang Wei Tan, Kai Zeng, Gao Cong, Yanzhao Qin, Andreas Pfadler, Zhengping Qian, Jingren Zhou, Jiangneng Li, and Bin Cui. 2021. Cardinality Estimation in DBMS: A Comprehensive Benchmark Evaluation. *Proc. VLDB Endow.* 15, 4 (2021), 752–765. https://doi.org/10.14778/3503585.3503586

[28] Venky Harinarayan, Anand Rajaraman, and Jeffrey D. Ullman. 1996. Implementing Data Cubes Efficiently. In *Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data, Montreal, Quebec, Canada, June 4-6, 1996*, H. V. Jagadish and Inderpal Singh Mumick (Eds.). ACM Press, 205–216. https://doi.org/10.1145/233269.233333

[29] Ching-Tien Ho, Rakesh Agrawal, Nimrod Megiddo, and Ramakrishnan Srikant. 1997. Range Queries in OLAP Data Cubes. In *SIGMOD 1997, Proceedings ACM SIGMOD International Conference on Management of Data, May 13-15, 1997, Tucson, Arizona, USA*, Joan Peckham (Ed.). ACM Press, 73–88. https://doi.org/10.1145/253260.253274

[30] Jorng-Tzong Horng, Yu-Jan Chang, and Baw-Jhiune Liu. 2003. Applying evolutionary algorithms to materialized view selection in a data warehouse. *Soft Comput.* 7, 8 (2003), 574–581. https://doi.org/10.1007/S00500-002-0243-1

[31] Alekh Jindal, Konstantinos Karanasos, Sriram Rao, and Hiren Patel. 2018. Selecting Subexpressions to Materialize at Datacenter Scale. *Proc. VLDB Endow.* 11, 7 (2018), 800–812. https://doi.org/10.14778/3192965.3192971

[32] Alekh Jindal, Konstantinos Karanasos, Sriram Rao, and Hiren Patel. 2018. Selecting Subexpressions to Materialize at Datacenter Scale. *Proc. VLDB Endow.* 11, 7 (2018), 800–812. https://doi.org/10.14778/3192965.3192971

[33] Alekh Jindal, Shi Qiao, Hiren Patel, Zhicheng Yin, Jieming Di, Malay Bag, Marc T. Friedman, Yifung Lin, Konstantinos Karanasos, and Sriram Rao. 2018. Computation Reuse in Analytics Job Service at Microsoft. In *Proceedings of the 2018 International Conference on Management of Data, SIGMOD Conference 2018, Houston, TX, USA, June 10-15, 2018*, Gautam Das, Christopher M. Jermaine, and Philip A. Bernstein (Eds.). ACM, 191–203. https://doi.org/10.1145/3183713.3190656

[34] Andreas Kipf, Thomas Kipf, Bernhard Radke, Viktor Leis, Peter Boncz, and Alfons Kemper. 2019. Learned Cardinalities: Estimating Correlated Joins with Deep Learning. In *9th Biennial Conference on Innovative Data Systems Research, CIDR 2019, Asilomar, CA, USA, January 13-16, 2019, Online Proceedings*. www.cidrdb.org. https://vldb.org/cidrdb/2019/learned-cardinalities-estimating-correlated-joins-with-deep-learning.html

[35] Eric Lambrecht, Subbarao Kambhampati, and Senthil Gnanaprakasam. 1999. Optimizing Recursive Information-Gathering Plans. In *Proceedings of the Sixteenth International Joint Conference on Artificial Intelligence, IJCAI 99, Stockholm, Sweden, July 31 - August 6, 1999. 2 Volumes, 1450 pages*, Thomas Dean (Ed.). Morgan Kaufmann, 1204–1211. http://ijcai.org/Proceedings/99-2/Papers/076.pdf

[36] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2015. How Good Are Query Optimizers, Really? *Proc. VLDB Endow.* 9, 3 (2015), 204–215. https://doi.org/10.14778/2850583.2850594

[37] Alon Y. Levy, Alberto O. Mendelzon, Yehoshua Sagiv, and Divesh Srivastava. 1995. Answering Queries Using Views. In *Proceedings of the Fourteenth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems, May 22-25, 1995, San Jose, California, USA*, Mihalis Yannakakis and Serge Abiteboul (Eds.). ACM Press, 95–104. https://doi.org/10.1145/212433.220198

[38] Alon Y. Levy, Anand Rajaraman, and Joann J. Ordille. 1996. Querying Heterogeneous Information Sources Using Source Descriptions. In *VLDB'96, Proceedings of 22th International Conference on Very Large Data Bases, September 3-6, 1996, Mumbai (Bombay), India*, T. M. Vijayaraman, Alejandro P. Buchmann, C. Mohan, and Nandlal L. Sarda (Eds.). Morgan Kaufmann, 251–262. http://www.vldb.org/conf/1996/P251.PDF

[39] Runzhong Li, Qilong Li, Haotian Liu, Rui Mao, Qing Li, and Bo Tang. 2025. Athena: An Effective Learning-based Framework for Query Optimizer Performance Improvement. *Proc. ACM Manag. Data* 3, 3 (2025), 132:1–132:24. https://doi.org/10.1145/3725395

[40] Xi Liang, Aaron J. Elmore, and Sanjay Krishnan. 2019. Opportunistic View Materialization with Deep Reinforcement Learning. *CoRR* abs/1903.01363 (2019). arXiv:1903.01363 http://arxiv.org/abs/1903.01363

[41] Spyros Ligoudistianos, Dimitri Theodoratos, and Timos K. Sellis. 1998. Experimental Evaluation of Data Warehouse Configuration Algorithms. In *Ninth International Workshop on Database and Expert Systems Applications, Vienna, Austria, August 24-28, 1998, Proceedings*, Roland R. Wagner (Ed.). IEEE Computer

Society, 218–223. https://doi.org/10.1109/DEXA.1998.707406

[42] Hoshi Mistry, Prasan Roy, S. Sudarshan, and Krithi Ramamritham. 2001. Materialized View Selection and Maintenance Using Multi-Query Optimization. In *Proceedings of the 2001 ACM SIGMOD international conference on Management of data, Santa Barbara, CA, USA, May 21-24, 2001*, Sharad Mehrotra and Timos K. Sellis (Eds.). ACM, 307–318. https://doi.org/10.1145/375663.375703

[43] Luis Leopoldo Perez and Christopher M. Jermaine. 2014. History-aware query optimization with materialized intermediate views. In *IEEE 30th International Conference on Data Engineering, Chicago, ICDE 2014, IL, USA, March 31 - April 4, 2014*, Isabel F. Cruz, Elena Ferrari, Yufei Tao, Elisa Bertino, and Goce Trajcevski (Eds.). IEEE Computer Society, 520–531. https://doi.org/10.1109/ICDE.2014.6816678

[44] Prasan Roy, S. Seshadri, S. Sudarshan, and Siddhesh Bhobe. 2000. Efficient and Extensible Algorithms for Multi Query Optimization. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data, May 16-18, 2000, Dallas, Texas, USA*, Weidong Chen, Jeffrey F. Naughton, and Philip A. Bernstein (Eds.). ACM, 249–260. https://doi.org/10.1145/342009.335419

[45] Yehoshua Sagiv and Mihalis Yannakakis. 1980. Equivalences Among Relational Expressions with the Union and Difference Operators. *J. ACM* 27, 4 (1980), 633–655. https://doi.org/10.1145/322217.322221

[46] Dimitri Theodoratos, Spyros Ligoudistianos, and Timos K. Sellis. 1999. Designing the Global Data Warehouse with SPJ Views. In *Advanced Information Systems Engineering, 11th International Conference CAiSE'99, Heidelberg, Germany, June 14-18, 1999, Proceedings (Lecture Notes in Computer Science)*, Matthias Jarke and Andreas Oberweis (Eds.), Vol. 1626. Springer, 180–194. https://doi.org/10.1007/3-540-48738-7_14

[47] Dimitri Theodoratos, Spyros Ligoudistianos, and Timos K. Sellis. 2001. View selection for designing the global data warehouse. *Data Knowl. Eng.* 39, 3 (2001), 219–240. https://doi.org/10.1016/S0169-023X(01)00041-6

[48] Dimitri Theodoratos and Timos K. Sellis. 1997. Data Warehouse Configuration. In *VLDB'97, Proceedings of 23rd International Conference on Very Large Data Bases, August 25-29, 1997, Athens, Greece*, Matthias Jarke, Michael J. Carey, Klaus R. Dittrich, Frederick H. Lochovsky, Pericles Loucopoulos, and Manfred A. Jeusfeld (Eds.). Morgan Kaufmann, 126–135. http://www.vldb.org/conf/1997/P126.PDF

[49] Dimitri Theodoratos and Timos K. Sellis. 1998. Data Warehouse Schema and Instance Design. In *Conceptual Modeling - ER '98, 17th International Conference on Conceptual Modeling, Singapore, November 16-19, 1998, Proceedings (Lecture Notes in Computer Science)*, Tok Wang Ling, Sudha Ram, and Mong-Li Lee (Eds.), Vol. 1507. Springer, 363–376. https://doi.org/10.1007/978-3-540-49524-6_29

[50] Odysseas G. Tsatalos, Marvin H. Solomon, and Yannis E. Ioannidis. 1996. The GMAP: A Versatile Tool for Physical Data Independence. *VLDB J.* 5, 2 (1996), 101–118. https://doi.org/10.1007/S007780050018

[51] Hidetoshi Uchiyama, Kanda Runapongsa, and Toby J. Teorey. 1999. A Progressive View Materialization Algorithm. In *DOLAP '99, ACM Second International Workshop on Data Warehousing and OLAP, November 6, 1999, Kansas City, Missouri, USA, Proceedings*, Il-Yeol Song and Toby J. Teorey (Eds.). ACM, 36–41. https://doi.org/10.1145/319757.319786

[52] Satyanarayana R. Valluri, Soujanya Vadapalli, and Kamalakar Karlapalem. 2002. Sprinkling Selections over Join DAGs for Efficient Query Optimization. *CoRR* cs.DB/0202035 (2002). https://arxiv.org/abs/cs/0202035

[53] Zhenrong Xu, Pengfei Wang, Guoze Xue, Qitong Yan, Shenghao Gong, Yelan Jiang, Yuren Mao, Yunjun Gao, Shu Shen, Wei Zhang, Dan Luo, and Lu Chen. 2024. UniView: A Unified Autonomous Materialized View Management System for Various Databases. *Proc. VLDB Endow.* 17, 12 (2024), 4353–4356. https://doi.org/10.14778/3685800.3685873

[54] Jian Yang, Kamalakar Karlapalem, and Qing Li. 1997. Algorithms for Materialized View Design in Data Warehousing Environment. In *VLDB'97, Proceedings of 23rd International Conference on Very Large Data Bases, August 25-29, 1997, Athens, Greece*, Matthias Jarke, Michael J. Carey, Klaus R. Dittrich, Frederick H. Lochovsky, Pericles Loucopoulos, and Manfred A. Jeusfeld (Eds.). Morgan Kaufmann, 136–145. http://www.vldb.org/conf/1997/P136.PDF

[55] Jingren Zhou, Per-Åke Larson, Johann Christoph Freytag, and Wolfgang Lehner. 2007. Efficient exploitation of similar subexpressions for query processing. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, Beijing, China, June 12-14, 2007*, Chee Yong Chan, Beng Chin Ooi, and Aoying Zhou (Eds.). ACM, 533–544. https://doi.org/10.1145/1247480.1247540